

Wheeler Memory

Open-Source Cellular Automaton Memory for LLMs

Build agentic systems that remember, forget, and reconstruct like humans do.

Why This Matters

The LLM Memory Problem

- **Vector DBs $\neq$ Memory:** Similarity search is not episodic recall. Same query = same result always.
- **Reconstruction is novel:** Human memory doesn't retrieve verbatim. Context reshapes recall.
- **Forgetting is useful:** Models without decay never reach epistemic humility. They hallucinate confidently.
- **Associative recall is missing:** Semantic search finds similar documents. CA finds related *concepts* through learned dynamics.

Status quo: We're treating databases like memory. Time for something different.

What You Can Build

Use Cases, Research & Production

Use Case	What's Possible
Agentic systems	Agents with persistent state, forgetting curves, contextual decision-making
Chatbots	Personalized conversation that remembers and forgets naturally
Semantic search	Query expansion via associative warmth (2-hop spreading activation)
Memory evaluation	Benchmark reconstruction fidelity, oscillation detection, temperature dynamics
Research	Test neuroscience theories via symbolic collapse (IIT connections)
Privacy-first LLMs	Local-only memory, no cloud, no data escape

No proprietary backend. Pure Python. Run anywhere.

Quick Start: 3 Commands

Install and Run

```
git clone https://github.com/fantomx42/wheeler-memory.git
cd wheeler-memory
pip install -e .
```

Launch the Dashboard

```
wheeler-ui
# Opens http://localhost:7437
```

Store and Recall a Memory

```
wheeler-store "remind me to debug the API timeout"
wheeler-recall "api performance issues" # finds it via semantic similarity
```

Done. No GPU required. Runs on CPU. Python 3.11+ only.

Two Recall Modes

Exact vs. Semantic

Exact Recall (Default)

```
wheeler-store "fix the python syntax error"  
wheeler-recall "fix the python syntax error" # exact match only
```

- Uses SHA-256 hashing
- Deterministic: same text always produces same fingerprint
- Fast, zero dependencies
- One word changes = completely different pattern

Semantic Recall (Install Extra)

```
pip install -e ".[embed]"  
wheeler-store --embed "fix the python syntax error"
```

The Science: Symbolic Collapse Model

What We're Actually Computing

Hypothesis: Meaning is what *survives symbolic pressure*.

```
Input text
  ↓
Dimensionality reduction (hash → 64×64)
  ↓
3-state CA evolution (local rules only)
  ↓
Convergence → Attractor (unique fixed point)
  ↓
Symbolic pressure: Information collapses to essential structure
```

Key insight: The attractor is not the text. It's what remains after lossy compression through dynamics.

Connections to IIT (Integrated Information Theory)

The 3-State CA Rule

How Attractors Form

```
# Simplified pseudocode
for each cell in grid:
    neighbors = von_neumann(cell)
    local_max = max(neighbors) > threshold
    local_min = min(neighbors) < threshold
    slope_uphill = uphill_direction_exists()

    if local_max:
        cell += 1 # push toward +1 (35% probability)
    elif local_min:
        cell -= 1 # push toward -1 (35% probability)
    elif slope_uphill:
        cell += sign(uphill_direction) # flow (20% probability)
```

- **Von Neumann neighborhood:** Only 4-connected (up/down/left/right)
- **Wrapping boundaries:** Grid is a torus (no edge artifacts)

Temperature & Forgetting

How Memories Decay

Formula

```
temp = base_from_hits × decay_from_time  
  
base_from_hits = min(1.0, 0.3 + 0.7 × (hit_count / 10))  
decay_from_time = 2 ^ (-days_since_last_access / 7)
```

Constants

- **Half-life:** 7 days (configurable)
- **Hit saturation:** 10 recalls (configurable)
- **Fresh memory baseline:** 0.3 (warm on arrival)

Temperature Tiers

Reconstructive Recall

The Context-Dependent Memory Mechanism

Formula

```
blend = (1 -  $\alpha$ ) × stored_attractor +  $\alpha$  × query_seed  
reconstructed = evolve_and_interpret(blend)
```

Default: $\alpha = 0.3$ (memory-dominant, 70% stored + 30% query)

Why This Works

- **Like human memory:** Loftus research shows recall is reconstructive, not retrieval
- **Prevents rigidity:** Same memory reconstructs differently for different queries
- **Preserves stability:** Memory-dominant bias means recall is mostly stable

Tuning

Validation: 19 Modules, 167+ Tests

What We Prove

Datasets

Dataset	Size	Domain
MBPP	10k problems	Python code snippets
SWE-bench	Software engineering tasks	Real-world repo operations
BABILong	Multi-hop reasoning	Long-context synthesis

Test Coverage

- **Dynamics:** CA convergence, rotation retry, oscillation detection
- **Storage:** Pearson correlation, chunking, metadata persistence
- **Reconstruction:** Blending fidelity, context-dependent variance

Architecture Deep Dive

7-Layer Stack

- | | |
|----------------------|--|
| 1. agent.py | ← Ollama integration, tool dispatch, chat loop |
| 2. chunking.py | ← Domain routing (code, hardware, science, general, ...) |
| 3. reconstruction.py | ← Blending, α parameter, re-evolution |
| 4. storage.py | ← Pearson correlation, recall ranking |
| 5. temperature.py | ← Decay formulas, tier logic |
| 6. polarity.py | ← Dual encoding, aversion attractors, decay count |
| 7. dynamics.py | ← CA rules, convergence detection, GPU dispatch |

Key Modules

Module	Responsibility
dynamics.py	3-state CA evolution, convergence logic
gpu_dynamics.py	HIP/ROCm CUDA kernels, CPU fallback
storage.py	Chunked storage, Pearson search

Extensibility Points

Pluggable Layers

Custom Hashing

```
from wheeler_memory import store_memory

# Replace SHA-256 with your own encoder
store_memory("my text", hash_fn=my_custom_encoder)
```

Custom Chunks

```
# Add a domain-specific chunk beyond the built-in 6
chunking.add_chunk("robotics", keywords=["servo", "motor", "ros", ...])
```

Custom CA Rules

```
# Extend dynamics.py with alternative evolution rules
```

Testing & Benchmarks

What the Test Suite Covers

Oscillation Detection

```
# Tests ensure we catch periodic cycles reliably  
brick = evolve(seed)  
assert brick.state in ["CONVERGED", "OSCILLATING", "CHAOTIC"]
```

Rotation Retry

```
# Same seed at 0°/90°/180°/270° should converge consistently  
for angle in [0, 90, 180, 270]:  
    brick = evolve(seed, rotation=angle)  
    assert brick.state != "CHAOTIC"
```

GPU Benchmarks

Web UI & CLI Ecosystem

10 CLI Tools

Tool	Purpose
<code>wheeler-store</code>	Save a memory
<code>wheeler-recall</code>	Find similar memories
<code>wheeler-ui</code>	Web dashboard
<code>wheeler-temps</code>	List all memories + temperatures
<code>wheeler-forget</code>	Delete a memory
<code>wheeler-sleep</code>	Consolidate stale memories
<code>wheeler-agent</code>	Start the Darman chatbot
<code>wheeler-info</code>	System info + config
<code>wheeler-scrub</code>	Visualize attractor formation

Integration Points

Ecosystem

Tool	Integration	Notes
Ollama	Native	<code>--ollama-host</code> flag in agent
Open WebUI	Pipeline function	Drop-in Wheeler memory layer
Hugging Face	Hub integration	Load/share memory archives
LangChain	Adapter (contrib)	Use Wheeler as memory backend
LlamaIndex	Vector store (planned)	Pluggable recall engine
FastAPI	HTTP API (demo)	Stateless recall endpoints

Running the Agent

```
wheeler-agent --ollama-host "http://localhost:11434"  
# Starts interactive chatbot with auto-recall and auto-store
```

Roadmap (6–12 Months)

Phase 1: Core Hardening (Now)

- ✓ Test suite expansion (done)
- ✓ GPU parity validation (done)
-  Documentation polish
-  Community outreach

Phase 2: Multimodal (Q3 2026)

- Image embedding (CLIP)
- Audio embeddings (wav2vec)
- Cross-modal reconstruction

Phase 3: Distributed (Q4 2026)

How to Contribute

High-Impact Areas

GPU Optimization

- Improve HIP/ROCm kernels for 64×64 grids
- Add METAL (Apple Silicon) support
- Optimize rotation retry parallelism

Multimodal Encoders

- CLIP integration for images
- Audio embeddings for speech
- Multimodal reconstruction pipeline

Distributed Protocols

Philosophy: "The Formula Is the Foundation"

Backward Compatibility is Sacred

Rule: The CA rule, convergence detection, and Pearson recall are load-bearing walls.

- Changes to `dynamics.py` affect *all* existing memories
- Changes to `hashing.py` require regeneration
- Changes to `storage.py`'s similarity function shift recall rankings

Consequence: Modifications are treated as breaking changes. Version bumps are semantic. Deprecation is documented.

Everything Else is Commentary

- UI can be replaced
- Agent prompt can be tweaked

Community & Support

GitHub: `fantomx42/wheeler-memory`

- **Issues:** Bug reports, feature requests, discussions
- **Discussions:** Q&A, architecture deep-dives, use cases
- **Contributing:** Pull request workflow, code review, CLA
- **Roadmap:** Public issue tracking, milestone planning

Documentation

- **Architecture:** Layer-by-layer walkthrough
- **API Reference:** Every class, function, parameter
- **CLI Guide:** All 10 tools with examples
- **Design Principles:** 7 axioms + rationale
- **Concepts:** Reconstruction theory, symbolic collapse, IIT

Call to Action

Start Contributing Today

```
# 1. Clone the repo
git clone https://github.com/fantomx42/wheeler-memory.git
cd wheeler-memory

# 2. Set up dev environment
pip install -e ".[dev]"
pytest tests/ -v

# 3. Check the roadmap
# See CONTRIBUTING.md for high-impact areas

# 4. Open a PR or discussion
```

Your Project Ideas

- **Chatbot:** Personal memory assistant

Next Steps

For Researchers

- Read `docs/concepts.md` (symbolic collapse model)
- Explore `docs/design.md` (7 principles)
- Open an issue with your research questions

For Practitioners

- Follow the [Installation Guide](#)
- Try the [Interactive Demo](#)
- Build something and share on Discussions

For Contributors

- Pick a [roadmap task](#)

"The formula is the foundation."

Star the repo. Read the docs. Build with us.

`github.com/fantomx42/wheeler-memory`

